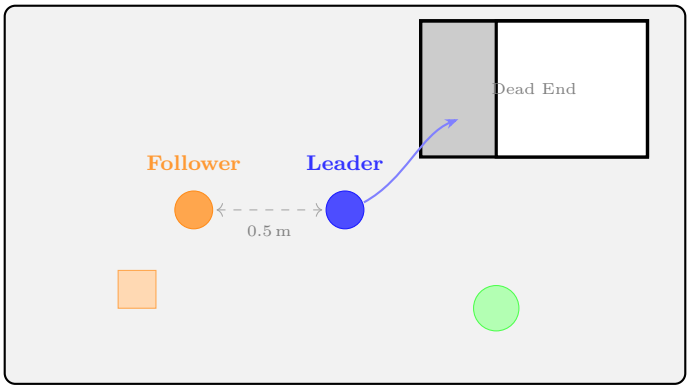
Model-Driven Engineering Lab

Leader-Follower Multi-Robot System

with ROS2, Gazebo, and Uppaal



Course:	Model-Driven Engineering
Hardware:	2× TurtleBot3 Burger (ROS2 Humble)
Tools:	Uppaal 5.x · ITEMIS Create · Gazebo Classic 11
Duration:	5 lab sessions (approx. 10–14 hours)
Deliverable:	Working simulation + Uppaal model + report

This assignment may be completed individually or in groups of two.
All code is written in Python for ROS2 Humble (Ubuntu 22.04).

Contents

1 Introduction and Scenario **2**
- 1.1 Motivation . . . 2
- 1.2 The Scenario . . . 2
- 1.3 Edge Cases to Handle . . . 2

2 Background: Model-Driven Engineering **4**
- 2.1 The MDE Workflow . . . 4
- 2.2 Tools Used . . . 4
- 2.3 Repository Structure . . . 4

3 Stage 1 — Requirements Analysis **5**
- 3.1 System Requirements . . . 5
  - 3.1.1 Functional Requirements . . . 5
  - 3.1.2 Safety Requirements . . . 5

4 Stage 2 — State Machine Modelling (ITEMIS Create) **6**
- 4.1 Provided Models . . . 6
- 4.2 ITEMIS Create Notation Summary . . . 6

5 Stage 3 — Timed Automata (Uppaal) **8**
- 5.1 What is a Timed Automaton? . . . 8
- 5.2 The Provided Uppaal Model . . . 8
- 5.3 Query Language . . . 8

6 Stage 4 — Property Verification Summary **11**

7 Stage 5 — Code Skeleton and Implementation **12**
- 7.1 Architecture Overview . . . 12
- 7.2 Custom Messages . . . 12
- 7.3 Building the Workspace . . . 13
- 7.4 Skeleton File Structure . . . 13

8 Stage 6 — Simulation Testing (Gazebo) **15**
- 8.1 Launching the Simulation . . . 15
- 8.2 Expected Startup Sequence . . . 15

9 Stage 7 — Real Robot Deployment (Bonus) **17**
- 9.1 Network Setup . . . 17
- 9.2 Per-Robot Bring-Up . . . 17

10 Deliverables and Assessment **18**
- 10.1 Deliverables . . . 18
- 10.2 Assessment Rubric . . . 18

Appendix A — Uppaal Quick Reference **19**

Appendix B — ROS2 Debugging Commands **19**

Appendix C — Key Parameters Reference **20**

1 Introduction and Scenario

1.1 Motivation

Modern software systems are rarely designed purely from intuition. *Model-Driven Engineering* (MDE) is a software development paradigm in which models—not code—are the primary artifacts of a project. By formalising behaviour through diagrams, timed automata, and verified models *before* writing a single line of executable code, we gain:

- Early detection of design flaws (deadlocks, livelocks, unreachable states);
- Unambiguous communication of system behaviour across the team;
- A clear, traceable path from abstract specification to concrete implementation;
- Confidence that the final system satisfies stated requirements.

In this lab you will follow the full MDE workflow for a real robotic application: you will *model*, *verify*, *generate*, *implement*, *simulate*, and finally *deploy* a cooperative multi-robot system.

1.2 The Scenario

You are given two TurtleBot3 Burger ground robots operating in an indoor environment. One robot acts as the **Leader**: it explores the environment using Simultaneous Localisation and Mapping (SLAM), building a map as it goes. The second robot acts as the **Follower**: it tracks the leader and stays approximately 0.5 m behind it at all times.

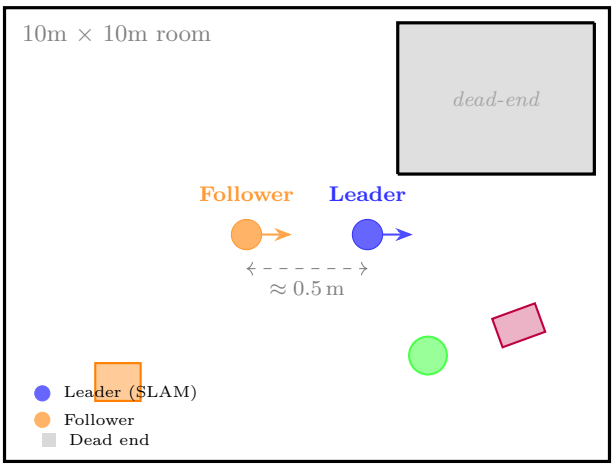


Figure 1: Top-down view of the exploration environment. The leader navigates into the dead-end corridor (NE quadrant); the follower must coordinate to resolve the situation.

1.3 Edge Cases to Handle

Real cooperative robot systems must be robust. The following *edge cases* are part of the formal specification and must be handled by your implementation:

EC1. Follower too far. The leader starts moving but the follower has not yet reached the correct following distance. The leader must wait while the follower approaches.

EC2. Leader hits a dead end. The leader detects that all forward paths are blocked by obstacles. It signals the follower to back up, reverses out, and resumes exploration. The follower temporarily drives forward to keep the path clear.

EC3. Follower loses the leader. Signal timeout or excessive distance causes the follower to lose track. The follower searches using the last known position; the leader halts and broadcasts its location until the follower recovers.

EC4. Both robots stuck (mutual wait). If both robots are simultaneously waiting for the other, the system must eventually time out and recover — not deadlock indefinitely.

2 Background: Model-Driven Engineering

2.1 The MDE Workflow

In MDE, a project proceeds through a series of increasingly concrete models, each derived from the previous one through *transformations*. Figure 2 shows the workflow for this lab.

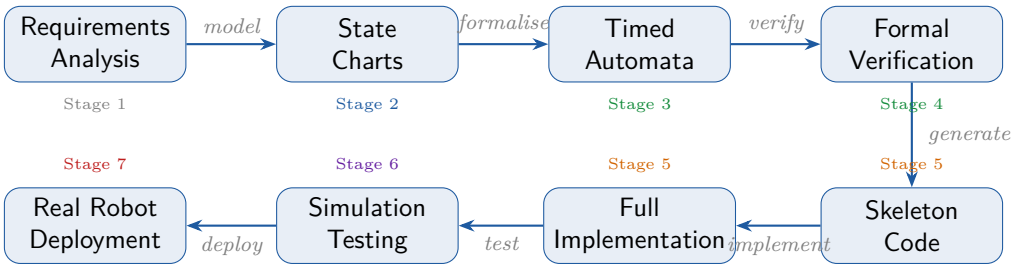


Figure 2: The MDE workflow followed in this lab. Each arrow represents a transformation.

2.2 Tools Used

Tool	Purpose	Stage
ITEMIS Create	Graphical statechart editor (export diagram as PNG/SVG)	2
Uppaal 5.x	Timed automata editor + model checker (free, uppaal.org)	3–4
ROS2 Humble	Robot operating system middleware	5–7
Gazebo Classic 11	Physics-based simulation	6
Nav2	Navigation stack (path planning)	5–7
slam_toolbox	SLAM for the leader robot	5–7
RViz2	Visualisation of map, robots, markers	6–7

2.3 Repository Structure

The starter repository is located at `/Users/.../ros_projects/leader_follower/` with the following layout:

```
leader_follower/
  models/
    statecharts/
      leader_statechart.ysc        # Leader ITEMIS Create model (your starting
           point)
      follower_statechart.ysc      # Follower ITEMIS Create model
    uppaal/
      leader_follower.xml          # Uppaal model (3 templates, 14 channels)
      leader_follower.q            # Verification queries
  src/
    leader_follower_msgs/          # Custom ROS2 messages (build this FIRST)
    simulation/                    # Gazebo world, launch files, RViz config
    leader_node/
      leader_node/
        leader_node_skeleton.py    # <-- YOUR starting point (Stage 5)
        leader_node.py             # Reference solution (DO NOT peek yet!)
    follower_node/
      follower_node/
        follower_node_skeleton.py # <-- YOUR starting point (Stage 5)
        follower_node.py           # Reference solution
    visualization_node/            # Provided: visualises robots in RViz2
```

3 Stage 1 — Requirements Analysis

Goal of this Stage

Understand the system requirements before writing any model or code.

3.1 System Requirements

The system must satisfy the following requirements.

3.1.1 Functional Requirements

FR1. The leader robot shall explore the environment autonomously using SLAM.

FR2. The follower robot shall maintain a distance of 0.5 ± 0.2 m behind the leader.

FR3. If the follower's distance exceeds 1.5 m, the leader shall pause and wait.

FR4. If the leader detects a dead end (all forward paths blocked), it shall signal the follower and execute a reversal manoeuvre.

FR5. If the follower loses contact with the leader (no status message for > 5 s), the follower shall execute a search procedure and the leader shall halt.

FR6. Both robots shall recover from every edge case without operator intervention, within the time bounds specified in the timed automata model.

3.1.2 Safety Requirements

SR1. The system shall *never* deadlock — at least one robot must always be able to make progress.

SR2. The follower shall not drive closer than 0.2 m to any obstacle.

SR3. The maximum time spent in any waiting state shall be bounded (see Table 1).

Table 1: State timeout bounds (encoded as clock invariants in the Uppaal model).

State	Robot	Max duration
`WAITING_FOR_FOLLOWER`	Leader	60 s
`DEAD_END`	Leader	15 s
`WAITING_FOR_LOST_FOLLOWER`	Leader	120 s
`APPROACHING`	Follower	60 s
`SEARCHING_FOR_LEADER`	Follower	90 s

Exercise 3.1 — Requirements Review

Before proceeding, answer the following questions in your lab report:

1. Which requirements are *safety* properties ("bad things never happen")? Which are *liveness* properties ("good things eventually happen")?
2. Describe in your own words what edge case EC4 (mutual wait) means in terms of robot behaviour. What would happen without the timeout bounds?
3. List any additional edge case you think the specification is missing. Justify your answer.

4 Stage 2 — State Machine Modelling (ITEMIS Create)

Goal of this Stage

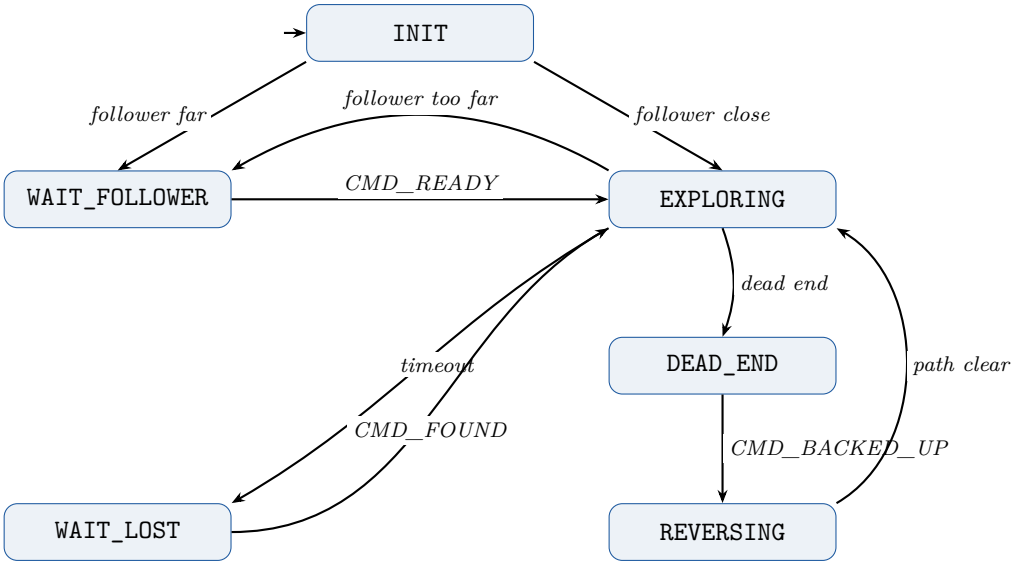
Create precise, readable statechart models for both robots. Statecharts are the bridge between informal requirements and formal timed automata.

4.1 Provided Models

Open the provided ITEMIS Create projects in the ITEMIS Create IDE (Eclipse-based; import via `File` → `Import` → `Existing Projects into Workspace`).

File: `models/statecharts/leader_statechart.ysc`

The leader statechart defines the following states and key transitions:



4.2 ITEMIS Create Notation Summary

Listing 1: ITEMIS Create Statechart Language (SCL) syntax (excerpt).

```
// Interface declarations (in the statechart specification)
@EventDriven
interface:
    in event follower_close     // incoming event
    in event dead_end_detected
    out event CMD_DEAD_END      // outgoing event
    var dist : real = 0.0       // variable

// Transitions inside the diagram are specified as:
//   trigger [guard] / action

// Entry transition (from Entry node to INIT)
// (graphical; no textual syntax needed)

// INIT --> EXPLORING on event with guard
follower_close [dist <= 1.0] / ;   // guard in brackets

// EXPLORING --> DEAD_END on event with action
dead_end_detected / raise CMD_DEAD_END;

// Timed transition: leave state after 60 seconds
```

```
after 60s / ;
```

Notation: `trigger [guard] / action` — guards are boolean expressions in square brackets; actions use `raise Interface.event` or variable assignments.

Exercise 4.1 — Statechart Inspection and Extension

1. Export both ITEMIS Create statecharts (leader + follower) as PNG or SVG. Include the exported diagrams in your report.
2. The follower statechart is missing a direct transition from `LEADING_TEMPORARILY` to `SEARCHING_FOR_LEADER` for the case where the leader goes silent during the reversal manoeuvre.
   a) Add this transition to `follower_statechart.ysc` in ITEMIS Create.
   b) Specify the guard condition and any actions on the new transition.
3. Propose and add one additional state to *either* robot's statechart that represents a scenario not yet modelled (e.g. battery low, sensor fault). Justify your choice and specify all new transitions.
4. Identify the *orthogonal region* in the follower statechart (communication submachine). What purpose does it serve?

5 Stage 3 — Timed Automata (Uppaal)

Goal of this Stage

Translate the statecharts into Uppaal timed automata, which allow formal verification. The key difference from statecharts: every state can have a *clock invariant* ("this state is exited within x seconds") and transitions can have *guards* (conditions on clocks and variables).

5.1 What is a Timed Automaton?

A Timed Automaton (TA) extends a finite-state automaton with:

- **Clocks** $x, y, \ldots \in \mathbb{R}_{\geq 0}$ — real-valued timers that increment continuously.
- **Clock invariants** on locations: e.g. $x \leq 15$ means "leave this state within 15 seconds."
- **Clock guards** on transitions: e.g. $x \geq 5$ enables a transition only after 5 seconds.
- **Resets**: $x := 0$ resets a clock on a transition.

A **network of timed automata** (NTA) is a set of TA running in parallel, synchronising via channels: `signal!` (sender) pairs with `signal?` (receiver).

5.2 The Provided Uppaal Model

Open `models/uppaal/leader_follower.xml` in Uppaal 5.x. The model contains three templates:

Template	Role
`LeaderRobot`	States + transitions of the leader; clock `t` with invariants
`FollowerRobot`	States + transitions of the follower; clock `t` with invariants
`Environment`	Non-deterministic event generator: obstacles, signal loss, distance changes

Global channel declarations (excerpt):

Listing 2: Global declarations in leader_follower.xml.

```
chan follower_ready;          // Follower -> Leader: in position
chan leader_start;            // Leader -> Follower: begin
broadcast chan dead_end_alert; // Leader -> Follower: dead end!
chan follower_backed_up;       // Follower -> Leader: backed up
chan reversal_done;            // Leader -> Follower: reversal complete
chan leader_lost_signal;       // Follower -> Leader: lost contact
chan leader_found_reply;       // Leader -> Follower: I am here
broadcast chan emergency_stop; // All -> All: halt
```

5.3 Query Language

Uppaal uses a subset of Computation Tree Logic (CTL):

Syntax	Reading	Type
`A[] p`	p always holds on all paths	safety
`E<> p`	p eventually holds on some path	reachability
`A<> p`	p eventually holds on all paths	(strong) liveness
`E[] p`	p always holds on some path	—
`p -> q`	whenever p, eventually q (leads-to)	liveness

The special predicate `deadlock` is true in a state where no transition is enabled (even after arbitrary time passage).

Exercise 5.1 — Loading and Running Queries

1. Open `leader_follower.xml` in Uppaal. Navigate to the *Simulator* tab and step through an execution to get a feel for the model.
2. Open `leader_follower.q` via *File* → *Open Queries*. Run all 14 queries from the *Verifier* tab.
3. Record your results in the following table (fill the Expected and Result columns):

#	Query	Type	Expected	Result
1	`A[] not deadlock`	safety	SAT	
2	`E<> (Leader.EXPLORING and Follower.FOLLOWING)`	reach.	SAT	
3	`Follower.APPROACHING --> Follower.FOLLOWING`	liveness	SAT	
4	`Leader.DEAD_END --> Leader.EXPLORING`	liveness	SAT	
5	`Follower.SEARCHING_FOR_LEADER --> Follower.APPROACHING`	liveness	SAT	
6	`E<> Leader.DEAD_END`	reach.	SAT	
7	`E<> Follower.LEADING_TEMPORARILY`	reach.	SAT	
8	no mutual timeout (clock bound)	safety	SAT	
9	`A[] (WAIT_FOLLOWER ⇒ t ≤ 60)`	safety	SAT	
10	`A[] (SEARCHING ⇒ t ≤ 90)`	safety	SAT	
11	progress from EXPLORING	liveness	SAT	
12	`A<> (Leader.EXPLORING and Follower.FOLLOWING)`	univ. live.	?	
13	mutual exclusion of conflicting states	safety	SAT	
14	follower leading ⇒ leader reversing/exploring	safety	SAT	

Exercise 5.2 — Analysing Query 12

Query 12 (`A<>` universal liveness) is marked with "?" because it may *not* be satisfied.

1. Run query 12. If it fails, Uppaal will show a *counter-example trace*. Use the simulator to step through it and identify which states are reached.
2. Explain in your own words why `A<>` is harder to satisfy than `E<>`.
3. The failure is caused by the `ERROR` states in both templates. Modify query 12 to *assume* neither robot is in `ERROR` and re-verify. Hint: use the `imply` operator or add a guard.
4. Add a new property of your own that is meaningful for this system (e.g. "after a dead end, reversal completes within 30 seconds"). Write the query, add it to `leader_follower.q`, and verify it. Explain the result.

Exercise 5.3 — Extending the Uppaal Model

1. The `Environment` template currently models distance changes by cycling through values 0–2 deterministically. Modify it so that the distance can also jump directly to 3 (LOST) at any time, simulating sudden signal loss. Re-verify the deadlock-freedom query. Does it still hold?

2. Add a `BatteryMonitor` template that:
   - maintains an integer variable `battery` ($100 \to 0$) that decreases over time;
   - sends an `emergency_stop` broadcast when `battery == 0`;
   - resets when `emergency_stop` is received (robot returns to dock).

   Write at least two new queries for the extended model.

6 Stage 4 — Property Verification Summary

Goal of this Stage

Consolidate your verification results and argue formally about key system properties. This stage is primarily a *report-writing* stage.

Exercise 6.1 — Verification Report

Write a concise section (1–2 pages) in your lab report that:

1. States the three most important properties for this system and justifies why.
2. For each property, gives:
   - The informal English description,
   - The formal Uppaal query,
   - The verification result (satisfied / not satisfied),
   - A brief argument for *why* it holds (or what assumption is needed).
3. Compares the properties verified here with what you would need for a *safety-critical* certification (e.g. ISO 26262 for automotive systems). What additional properties would you need to verify?

7 Stage 5 — Code Skeleton and Implementation

Goal of this Stage

Translate the verified formal model into executable ROS2 Python code. You will fill in the state machine logic in the provided skeleton files.

7.1 Architecture Overview

The system runs as five cooperating ROS2 nodes:

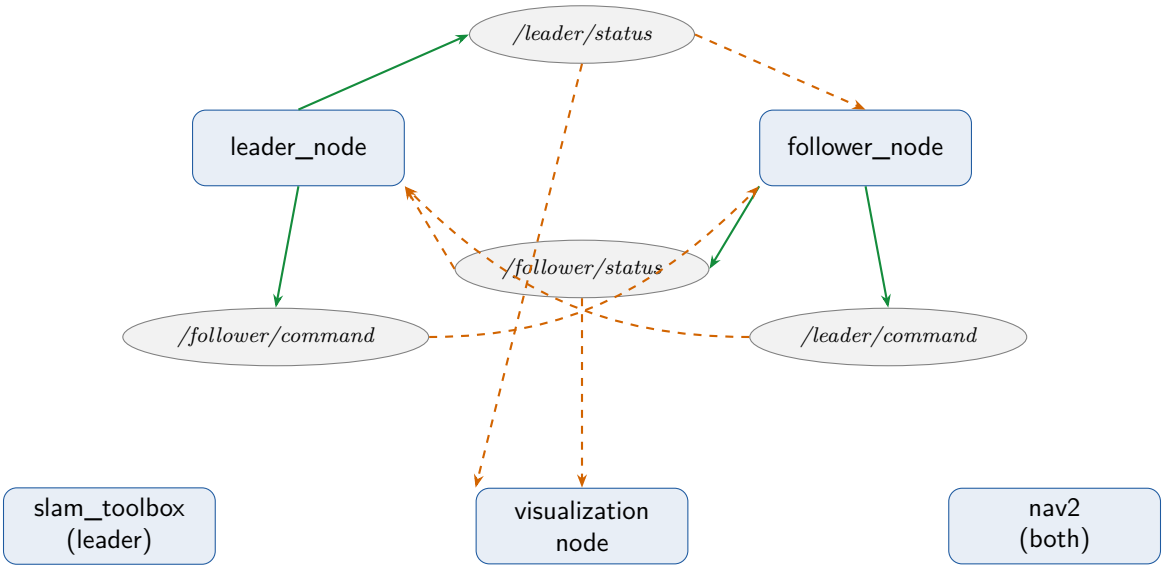


Figure 3: ROS2 node communication graph. Green solid = publish; orange dashed = subscribe.

7.2 Custom Messages

Two custom message types are defined in `src/leader_follower_msgs/msg/`:

Listing 3: RobotStatus.msg — broadcast by each robot at 10 Hz.

```
std_msgs/Header header
uint8  state           # current state enum value
string state_name      # human-readable state name
float64 x, y          # position in map frame (from odometry)
float64 heading       # yaw angle in radians
float64 distance_to_other  # Euclidean distance to other robot

uint8 STATE_INIT=0    uint8 STATE_EXPLORING=2   uint8 STATE_DEAD_END=5
uint8 STATE_REVERSING=6     uint8 STATE_WAITING_FOR_LOST_FOLLOWER=11
uint8 STATE_FOLLOWING=3     uint8 STATE_APPROACHING=4
# ...
```

Listing 4: CoordinationCommand.msg — sent on state transitions.

```
std_msgs/Header header
uint8  command    # one of the CMD_* constants below
string data       # optional payload (e.g., "x,y" for I_AM_HERE)

uint8 CMD_READY=0   uint8 CMD_DEAD_END=7   uint8 CMD_BACKED_UP=11
uint8 CMD_RESUME=2  uint8 CMD_LOST_YOU=9   uint8 CMD_I_AM_HERE=6
```

```
# ...
```

7.3 Building the Workspace

```
cd ~/ros_projects/leader_follower
export TURTLEBOT3_MODEL=burger
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp
colcon build --symlink-install
source install/setup.bash
```

Important

Always build `leader_follower_msgs` first (colcon handles this automatically via dependency resolution). If you encounter import errors, run `colcon build -packages-select leader_follower_msgs` first.

7.4 Skeleton File Structure

Open `src/leader_node/leader_node/leader_node_skeleton.py`. Every `# TODO` comment corresponds to a state handler that you must implement. The helper methods (LiDAR sector analysis, `_stop()`, `_transition_to()`, `_send_command_to_follower()`) are fully provided — use them.

Exercise 7.1 — Implement the Leader Node

Fill in the following methods in `leader_node_skeleton.py`:

1. **`_handle_init()`** — Check if follower status has been received; transition to `EXPLORING` (close) or `WAITING_FOR_FOLLOWER` (far).
2. **`_handle_exploring()`** — This is the core handler. Implement the following priority chain:
   i. If follower is not connected (`_follower_is_connected()` returns False) → transition to `WAITING_FOR_LOST_FOLLOWER`.
   ii. If `follower_distance > FOLLOW_DISTANCE_FAR` → stop robot, transition to `WAITING_FOR_FOLLOWER`.
   iii. If `_is_dead_end()` returns True for `DEAD_END_CONFIRM_SCANS` consecutive calls → send `CMD_DEAD_END`, transition to `DEAD_END`.
   iv. Otherwise → publish a forward `Twist` (`linear.x = EXPLORE_LINEAR`).
3. **`_handle_dead_end()`** — Stop robot. Re-broadcast `CMD_DEAD_END` every `DEAD_END_BROADCAST_INTERVAL` seconds. When `self.dead_end_ack_received` is True (set by the callback), transition to `REVERSING`. If `_seconds_in_state() > DEAD_END_WAIT_TIMEOUT`, force transition to `REVERSING`.
4. **`_handle_reversing()`** — Publish a backward Twist (`linear.x = LINEAR_REVERSE_SPEED`). When the forward sector minimum range exceeds `REVERSAL_CLEAR_RANGE`, stop, send `CMD_RESUME`, and transition to `EXPLORING`.
5. **`_on_follower_ready()`**, **`_on_follower_lost()`**, **`_on_follower_found()`** — Event handlers called by the command callback. Implement the state transitions for each event.

Exercise 7.2 — Implement the Follower Node

Fill in the following methods in `follower_node_skeleton.py`:

1. **`_handle_init()`** — Wait for first leader status message; transition to `APPROACHING`.

2. **`_handle_approaching()`** — Drive toward a point `TARGET_DISTANCE` behind the leader's published pose. Use the provided `_compute_approach_goal()` to get the target position. Transition to `FOLLOWING` (sending `CMD_READY`) when `leader_distance` $\leq$ `APPROACH_THRESHOLD`.
3. **`_handle_following()`** — Implement the proportional controller:

$$v = K_{p,v} \cdot (d - d^*), \qquad \omega = K_{p,\omega} \cdot \beta$$

   where d is `self.leader_distance`, $d^* = 0.5\,\mathrm{m}$ (`TARGET_DISTANCE`), and β is `self.leader_bearing`. Clamp $v \in [-v_{\max}, v_{\max}]$ and $\omega \in [-\omega_{\max}, \omega_{\max}]$. Apply the safety stop: if the forward LiDAR minimum is $< 0.20\,\mathrm{m}$, set $v = 0$.
4. **`_handle_backing_up()`** — Drive backward at `BACKUP_SPEED`. Track distance backed up using the current pose relative to `self.backup_start_x`, `self.backup_start_y`. When `distance_backed_up` $\geq$ `BACKUP_DISTANCE`, send `CMD_BACKED_UP` and transition to `LEADING_TEMPORARILY`.
5. **`_handle_searching()`** — Implement at least Phase 0 of the search algorithm (navigate to `last_known_leader_x/y`). The heartbeat timer broadcasts `CMD_LOST_YOU` automatically. Transition to `APPROACHING` when a new leader status is received (this is already wired in `_leader_status_callback`).

8 Stage 6 — Simulation Testing (Gazebo)

Goal of this Stage

Validate your implementation in the Gazebo simulation environment before touching real hardware.

8.1 Launching the Simulation

```
# Terminal 1: start simulation (Gazebo + Nav2 + SLAM + all nodes)
source install/setup.bash
ros2 launch simulation simulation.launch.py

# Terminal 2: monitor robot states
ros2 topic echo /leader/status --field state_name
ros2 topic echo /follower/status --field state_name

# Terminal 3: watch coordination commands
ros2 topic echo /follower/command
ros2 topic echo /leader/command
```

8.2 Expected Startup Sequence

1. Gazebo loads `exploration_world.world`; both robots spawn.
2. Leader enters `INIT`, follower enters `INIT`.
3. Follower transitions to `APPROACHING`, drives toward leader.
4. Follower sends `CMD_READY`; leader transitions to `EXPLORING`.
5. Both enter cooperative mode: `EXPLORING` / `FOLLOWING`.
6. Leader navigates into the dead-end corridor (NE quadrant).
7. Leader detects dead end, sends `CMD_DEAD_END`.
8. Follower transitions to `BACKING_UP`, then `LEADING_TEMPORARILY`.
9. Leader reverses; clears the corridor; sends `CMD_RESUME`.
10. Both return to `EXPLORING` / `FOLLOWING`.

Exercise 8.1 — Observe and Document Normal Operation

1. Launch the simulation. Take a screenshot of RViz2 showing both robots and the growing SLAM map. Include it in your report.
2. Record a sequence of state transitions from the terminal output. Do they match the expected startup sequence above?
3. Measure the time from simulation start until both robots first reach cooperative mode (`EXPLORING`/`FOLLOWING`). Compare with the timeout bounds in Table 1.

Exercise 8.2 — Test Edge Case EC2: Dead End Recovery

1. Wait until the leader naturally navigates into the dead-end corridor, *or* use teleop to manually drive the leader into the corridor:

```
ros2 run turtlebot3_teleop teleop_keyboard \
  --ros-args --remap /cmd_vel:=/leader/cmd_vel
```

2. Observe the state transitions in both robots. Record the exact sequence of `state_name` values published on `/leader/status` and `/follower/status`.
3. Does the dead-end recovery protocol complete within the timeout specified in Table 1? Time it.
4. Take a screenshot of RViz2 showing the connection line colour change during recovery (orange = dead-end recovery in progress).

Exercise 8.3 — Test Edge Case EC3: Follower Lost

1. While both robots are in cooperative mode, kill the follower node:

```
# Find the follower node's PID and kill it
ros2 node list
kill $(ros2 node info /follower/follower_node | grep Pid | awk '{
    print $2}')
# Or simply Ctrl+C in the terminal running follower_node
```

2. Observe: the leader should transition to `WAITING_FOR_LOST_FOLLOWER` within 5 seconds and begin broadcasting `CMD_I_AM_HERE`.
3. Restart the follower node:

```
ros2 run follower_node follower_node
```

4. Verify that the follower re-enters `APPROACHING` → `FOLLOWING` and the leader resumes `EXPLORING`.
5. Document the recovery time (seconds from follower restart to cooperative mode).

Exercise 8.4 — Tuning the Controller (Advanced)

The follower's proportional controller gains are defined as constants at the top of `follower_node.py`:

```
KP_LINEAR  = 0.35    # linear velocity proportional gain
KP_ANGULAR = 0.90    # angular velocity proportional gain
TARGET_DISTANCE = 0.50  # desired gap (meters)
```

1. What happens if `KP_LINEAR` is too large? Too small? Test both.
2. What happens if `TARGET_DISTANCE` is set to 0.2 m? Does the safety stop activate? Why?
3. Find a combination of gains that minimises overshoot while maintaining smooth following. Plot the distance over time (use `ros2 topic echo` and record to CSV, then plot in Python/Matlab).

9 Stage 7 — Real Robot Deployment (Bonus)

Goal of this Stage

Deploy the validated simulation code on real TurtleBot3 Burger hardware. **This stage is optional** but strongly recommended for full marks.

Hardware Safety Rules

- Always keep a hand close to the e-stop button.
- Run first tests in a large, clear space with no people nearby.
- Ensure both robots have at least 50 % battery before starting.
- Set `MAX_LINEAR_VEL = 0.10` for first tests (half the default).

9.1 Network Setup

Both robots and the workstation must share the same ROS2 DDS domain:

```
# On ALL devices (workstation, leader Pi, follower Pi):
export ROS_DOMAIN_ID=42
export RMW_IMPLEMENTATION=rmw_cyclonedds_cpp

# Verify discovery:
ros2 node list    # should show nodes from all devices
```

9.2 Per-Robot Bring-Up

```
# On Leader Raspberry Pi:
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_bringup robot.launch.py

# On Follower Raspberry Pi:
export TURTLEBOT3_MODEL=burger
ros2 launch turtlebot3_bringup robot.launch.py

# On Workstation:
ros2 launch simulation real_robot.launch.py
```

Exercise 9.1 — Real Robot Validation

1. Verify that `ros2 topic list` shows `/leader/scan`, `/follower/scan`, `/leader/odom`, and `/follower/odom` on the workstation.
2. Place the robots in the correct start positions (0.5 m apart, facing the same direction). Observe the initial approach sequence.
3. If the follower oscillates (overshoot), reduce `KP_LINEAR`. If it is too slow to track, increase it.
4. Demonstrate the dead-end recovery by placing a large box in front of the leader while both robots are in cooperative mode.
5. Record a video of the full demo (initial approach + exploration + at least one edge case recovery). Submit alongside your report.

10 Deliverables and Assessment

10.1 Deliverables

Submit a single ZIP archive containing:

1. **Lab Report** (PDF, max 15 pages) structured as follows:
   - Stage 1: Requirement classification table + EC analysis
   - Stage 2: Rendered statecharts (original + your modifications)
   - Stage 3–4: Verification table + property arguments
   - Stage 5: Code diff (only your changes to the skeleton files)
   - Stage 6: Screenshots, state transition logs, edge case results
   - (Stage 7): Video link or attached MP4
2. **Modified model files:** `leader_statechart.ysc`, `follower_statechart.ysc`, `leader_follower.xml`, `leader_follower.q`
3. **Implemented node files:** `leader_node_skeleton.py`, `follower_node_skeleton.py` (renamed to `leader_node_impl.py`, `follower_node_impl.py`)

10.2 Assessment Rubric

Criterion	Evidence	Weight	Max pts
Stage 1: Req. analysis quality	Report: classification, EC4 explanation	10 %	10
Stage 2: Statechart correctness	Rendered diagrams, added transition + state	15 %	15
Stage 3: Uppaal query results	Completed verification table, correct results	15 %	15
Stage 4: Property arguments	Written justifications for top-3 properties	10 %	10
Stage 5a: Leader implementation	All 5 handler methods correct	15 %	15
Stage 5b: Follower implementation	All 5 handler methods correct	15 %	15
Stage 6: Simulation validation	Screenshots, state logs, edge case demos	10 %	10
Stage 6 (adv.): Controller tuning	Distance plot, gain analysis	5 %	5
Stage 7 (bonus): Real robot demo	Video showing all edge cases on hardware	+15 %	15
Total (without bonus)		100 %	100
Total (with bonus)		115 %	115

Academic Integrity

You may discuss concepts and debugging strategies with other groups, but all model files, code, and written analysis must be your own work. Any use of AI code-generation tools must be declared and the generated portions clearly identified. Undeclared AI-generated code will be treated as plagiarism.

Appendix A — Uppaal Quick Reference

A.1 Location Types

Type	Meaning
Normal	Standard location; time may pass.
Initial	Marked with a double circle; starting location.
Committed	Marked with `C`; no time may pass; must take a transition immediately.
Urgent	Marked with `U`; no time may pass, but can be pre-empted by other automata.

A.2 Transition Labels

Each edge can carry up to four labels (all optional):

Label	Example	Meaning
Select	`i: int[0,3]`	Non-deterministic binding
Guard	`x >= 5`	Condition on clocks/variables
Synchronisation	`signal! / signal?`	Send/receive on channel
Update	`x = 0, count++`	Side effects on transition

A.3 Query Operators Cheat Sheet

```
A[] not deadlock                        // Safety: deadlock freedom
A[] P.safe_state imply x <= 10         // Clock bound invariant
E<> P.goal_state                       // Reachability
A<> P.goal_state                       // Universal liveness
P.state1 --> P.state2                  // Leads-to (liveness)
A[] not (P.s1 and Q.s2)                // Mutual exclusion
```

A.4 Commonly Used Clock Patterns

```
// Pattern: time-bounded state (add invariant to location)
// Location "WAITING" has invariant: t <= 60
// Reset clock on entry:
trans S_PREV -> WAITING { update: t = 0 }
// Force exit after timeout:
trans WAITING -> ERROR { guard: t >= 60 }

// Pattern: periodic re-broadcast every N seconds
trans IDLE -> SEND  { guard: t >= N; update: t = 0 }
trans SEND  -> IDLE { update: action() }
```

Appendix B — ROS2 Debugging Commands

```
# List all active nodes
ros2 node list

# Show all topics with types
ros2 topic list -t

# Monitor state transitions (10 Hz)
ros2 topic echo /leader/status --field state_name
```

```
ros2 topic echo /follower/status --field state_name

# Record a bag for later analysis
ros2 bag record /leader/status /follower/status \
  /leader/command /follower/command /leader/scan /follower/scan

# Check node graph
ros2 run rqt_graph rqt_graph

# Manual dead-end test (teleop leader into corridor)
ros2 run turtlebot3_teleop teleop_keyboard \
  --ros-args --remap /cmd_vel:=/leader/cmd_vel

# Verify Nav2 is running
ros2 action list
# Should include: /leader/navigate_to_pose, /follower/navigate_to_pose

# Check transform tree
ros2 run tf2_tools view_frames
```

Appendix C — Key Parameters Reference

Parameter	Meaning	Default	File
`TARGET_DISTANCE`	Ideal follower-to-leader gap	0.50 m	`follower_node.py`
`APPROACH_THRESHOLD`	Close enough → switch to FOLLOWING	0.70 m	`follower_node.py`
`FAR_THRESHOLD`	Too far → re-approach	1.50 m	both
`DEAD_END_RANGE`	LiDAR range = blocked	0.40 m	`leader_node.py`
`REVERSAL_CLEAR_RANGE`	Forward clear → stop reversing	0.80 m	`leader_node.py`
`BACKUP_DISTANCE`	Follower reversal depth	1.50 m	`follower_node.py`
`KP_LINEAR`	Following linear gain	0.35	`follower_node.py`
`KP_ANGULAR`	Following angular gain	0.90	`follower_node.py`
`FOLLOWER_TIMEOUT`	No msg → lost (leader side)	5.0 s	`leader_node.py`
`LEADER_TIMEOUT`	No msg → lost (follower side)	5.0 s	`follower_node.py`
`DEAD_END_CONFIRM_SCANS`	Scans to confirm dead end	3	`leader_node.py`
`SEARCH_TIMEOUT`	Max search time	90.0 s	`follower_node.py`